



МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования

**«Дальневосточный федеральный университет»
(ДФУ)**

**ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ
ТЕХНОЛОГИЙ**

Лабораторная работа 4

Дисциплина «Теория вероятностей и математическая статистика»

Студент группы Б9123-01.03.02ии

Есакова Елизавета Михайловна

г. Владивосток

2025

ЛР №4 | Метод bootstrap

Целью данной лабораторной работы является изучение метода bootstrap.

https://colab.research.google.com/drive/1-b8tOyBxD5IzCsIu4C2yebQniV5al1np#scrollTo=GxtztI6iS_u

Метод bootstrap

Имеется выборка $X_1, X_2, \dots, X_n$ из X .

Для вычисления доверительного интервала параметра θ с уровнем доверия γ можно использовать алгоритм:

- 1 Найти оценку $\hat{\theta}_n$ параметра θ ;
- 2 Сгенерировать k новых выборок $X_1^i, X_2^i, \dots, X_n^i$ ($i = \overline{1, k}$), где X_j^i – случайный элемент исходной выборки;
- 3 Вычислить $\theta_i^* = \hat{\theta}_n(X_1^i, X_2^i, \dots, X_n^i)$ для $i = \overline{1, k}$;
- 4 Доверительный интервал $\left(\theta_{(\lfloor \frac{1-\gamma}{2} k \rfloor)}^*, \theta_{(\lceil \frac{1+\gamma}{2} k \rceil)}^* \right)$.

Собственная реализация в питоне:

```
def bootstrap(sample, estimator, doverie=0.95, n_bootstrap=1000):
    n = len(sample)

    bootstrap_estimates = []

    for _ in range(n_bootstrap):
        bootstrap_sample = np.random.choice(sample, size=n, replace=True)
        estimate = estimator(bootstrap_sample)
        bootstrap_estimates.append(estimate)

    alpha = (1 - doverie) / 2
    L_p = alpha * 100
    U_p = (1 - alpha) * 100

    L_x = np.percentile(bootstrap_estimates, L_p)
    U_x = np.percentile(bootstrap_estimates, U_p)

    return L_x, U_x
```

`doverie=0.95` - уровень доверия для доверительного интервала

Код для оценки методом bootstrap всех параметров распределений на основе выборок из ЛР №1 со своим методом бутстрап:

```
import numpy as np
```

```

from scipy import stats
from tabulate import tabulate

def bootstrap(sample, estimator, doverie=0.95, n_bootstraps=1000):
    n = len(sample)

    bootstrap_estimates = []

    for _ in range(n_bootstraps):
        bootstrap_sample = np.random.choice(sample, size=n, replace=True)
        estimate = estimator(bootstrap_sample)
        bootstrap_estimates.append(estimate)

    alpha = (1 - doverie) / 2
    L_p = alpha * 100
    U_p = (1 - alpha) * 100

    L_x = np.percentile(bootstrap_estimates, L_p)
    U_x = np.percentile(bootstrap_estimates, U_p)

    return L_x, U_x

uniform_params = {'loc': 5, 'scale': 10}
bernoulli_params = {'p': 0.7}
binomial_params = {'n': 20, 'p': 0.4}
normal_params = {'loc': 5, 'scale': 2}

np.random.seed(4)

samples = {
    'Uniform_100': stats.uniform.rvs(size=100, **uniform_params),
    'Uniform_1000': stats.uniform.rvs(size=1000, **uniform_params),
    'Bernoulli_100': stats.bernoulli.rvs(size=100, **bernoulli_params),
    'Bernoulli_1000': stats.bernoulli.rvs(size=1000, **bernoulli_params),
    'Binomial_100': stats.binom.rvs(size=100, **binomial_params),
    'Binomial_1000': stats.binom.rvs(size=1000, **binomial_params),
    'Normal_100': stats.norm.rvs(size=100, **normal_params),
    'Normal_1000': stats.norm.rvs(size=1000, **normal_params)
}

def estimate_a(sample):
    return np.min(sample)

def estimate_b(sample):
    return np.max(sample)

def estimate_ber_p(sample):

```

```

    return np.mean(sample)

def estimate_bin_p(sample):
    return np.mean(sample) / binomial_params['n']

def estimate_bin_n(sample):
    mean = np.mean(sample)
    p = estimate_bin_p(sample)
    return mean / p

def estimate_u(sample):
    return np.mean(sample)

def estimate_sigma(sample):
    return np.var(sample, ddof=1)

doveriie = 0.95
n_bootstraps = 1000

results = []

for name, sample in samples.items():
    if "Uniform" in name:
        a_bot = bootstrap(sample, estimate_a, doveriie, n_bootstraps)
        b_bot = bootstrap(sample, estimate_b, doveriie, n_bootstraps)
        results.append([name, "a", "min(sample)", uniform_params['loc'],
a_bot])
        results.append([name, "b", "max(sample)", uniform_params['loc'] +
uniform_params['scale'], b_bot])
    elif "Bernoulli" in name:
        p_bot = bootstrap(sample, estimate_ber_p, doveriie, n_bootstraps)
        results.append([name, "p", "mean(sample)", bernoulli_params['p'],
p_bot])
    elif "Binomial" in name:
        p_bot = bootstrap(sample, estimate_bin_p, doveriie, n_bootstraps)
        n_bot = bootstrap(sample, estimate_bin_n, doveriie, n_bootstraps)
        results.append([name, "p", "mean(sample)/n", binomial_params['p'],
p_bot])
        results.append([name, "n", "mean(sample)/ p",
binomial_params['n'], n_bot])
    elif "Normal" in name:
        u_bot = bootstrap(sample, estimate_u, doveriie, n_bootstraps)
        sigma2_bot = bootstrap(sample, estimate_sigma, doveriie,
n_bootstraps)
        results.append([name, "u", "mean(sample)", normal_params['loc'],
u_bot])
        results.append([name, "sigma^2", "var(sample)",
normal_params['scale']**2, sigma2_bot])

```

```
headers = ["Выборка", "Параметр", "Точечная оценка", "Значение",
"Bootstrap"]
print(tabulate(results, headers=headers, tablefmt="fancy_grid"))
```

Вывод таблицы, используя `tablefmt="fancy_grid"`:

Выборка	Параметр	Точечная оценка	Значение	Bootstrap
Uniform_100	a	min(sample)	5	(np.float64(5.062302552045899), np.float64(5.485341364195525))
Uniform_100	b	max(sample)	15	(np.float64(14.670298390136766), np.float64(14.834006771753128))
Uniform_1000	a	min(sample)	5	(np.float64(5.00906670215289), np.float64(5.063239372654827))
Uniform_1000	b	max(sample)	15	(np.float64(14.93488850985824), np.float64(14.993640396351307))
Bernoulli_100	p	mean(sample)	0.7	(np.float64(0.51), np.float64(0.7))
Bernoulli_1000	p	mean(sample)	0.7	(np.float64(0.683), np.float64(0.7390249999999999))
Binomial_100	p	mean(sample)/n	0.4	(np.float64(0.40099999999999997), np.float64(0.449))
Binomial_100	n	mean(sample)/ p	20	(np.float64(20.0), np.float64(20.0))
Binomial_1000	p	mean(sample)/n	0.4	(np.float64(0.39484625), np.float64(0.4080525))
Binomial_1000	n	mean(sample)/ p	20	(np.float64(20.0), np.float64(20.0))
Normal_100	u	mean(sample)	5	(np.float64(4.260231542503547), np.float64(5.035014677902269))
Normal_100	sigma^2	var(sample)	4	(np.float64(2.898043317847941), np.float64(5.25728592322289))
Normal_1000	u	mean(sample)	5	(np.float64(4.888915189312089), np.float64(5.136528261006682))
Normal_1000	sigma^2	var(sample)	4	(np.float64(3.7160220075387596), np.float64(4.366522685047057))

Код для оценки методом bootstrap всех параметров распределений на основе выборок из ЛР №1 с методом бутстрап из `scipy.stats`:

```
import numpy as np
from scipy.stats import bootstrap
from scipy import stats
from tabulate import tabulate

uniform_params = {'loc': 5, 'scale': 10}
bernoulli_params = {'p': 0.7}
binomial_params = {'n': 20, 'p': 0.4}
normal_params = {'loc': 5, 'scale': 2}

np.random.seed(4)

samples = {
    'Uniform_100': stats.uniform.rvs(size=100, **uniform_params),
    'Uniform_1000': stats.uniform.rvs(size=1000, **uniform_params),
    'Bernoulli_100': stats.bernoulli.rvs(size=100, **bernoulli_params),
    'Bernoulli_1000': stats.bernoulli.rvs(size=1000, **bernoulli_params),
    'Binomial_100': stats.binom.rvs(size=100, **binomial_params),
    'Binomial_1000': stats.binom.rvs(size=1000, **binomial_params),
    'Normal_100': stats.norm.rvs(size=100, **normal_params),
```

```

        'Normal_1000': stats.norm.rvs(size=1000, **normal_params)
    }

    def estimate_a(sample):
        return np.min(sample)

    def estimate_b(sample):
        return np.max(sample)

    def estimate_ber_p(sample):
        return np.mean(sample)

    def estimate_bin_p(sample):
        return np.mean(sample) / binomial_params['n']

    def estimate_bin_n(sample):
        mean = np.mean(sample)
        p = estimate_bin_p(sample)
        return mean / p

    def estimate_u(sample):
        return np.mean(sample)

    def estimate_sigma(sample):
        return np.var(sample, ddof=1)

doveriie = 0.95
results = []

for name, sample in samples.items():
    if "Uniform" in name:

        res_a = bootstrap((sample,), estimate_a,
confidence_level=doveriie, n_resamples=1000)
        res_b = bootstrap((sample,), estimate_b,
confidence_level=doveriie, n_resamples=1000)
        results.append([name, "a", uniform_params['loc'],
res_a.confidence_interval])
        results.append([name, "b", uniform_params['loc'] +
uniform_params['scale'], res_b.confidence_interval])

    elif "Bernoulli" in name:
        res_p = bootstrap((sample,), estimate_ber_p,
confidence_level=doveriie, n_resamples=1000)
        results.append([name, "p", bernoulli_params['p'],
res_p.confidence_interval])

    elif "Binomial" in name:
        res_p = bootstrap((sample,), estimate_bin_p,
confidence_level=doveriie, n_resamples=1000)

```

```

        results.append([name, "p", binomial_params['p'],
res_p.confidence_interval])
        res_n = bootstrap((sample,), estimate_bin_n,
confidence_level=doveriie, n_resamples=1000)
        results.append([name, "n", binomial_params['n'],
res_n.confidence_interval])

    elif "Normal" in name:
        res_u = bootstrap((sample,), estimate_u,
confidence_level=doveriie, n_resamples=1000)
        res_sigma2 = bootstrap((sample,), estimate_sigma,
confidence_level=doveriie, n_resamples=1000)
        results.append([name, "u", normal_params['loc'],
res_u.confidence_interval])
        results.append([name, "sigma^2", normal_params['scale']**2,
res_sigma2.confidence_interval])

headers = ["Выборка", "Параметр", "Значение", "Bootstrap scipy"]
print(tabulate(results, headers=headers, tablefmt="fancy_grid"))

```

Вывод таблицы:

Выборка	Параметр	Значение	Bootstrap scipy
Uniform_100	a	5	ConfidenceInterval(low=np.float64(5.062302552045899), high=np.float64(5.08986097667555))
Uniform_100	b	15	ConfidenceInterval(low=np.float64(14.762744547762418), high=np.float64(14.834006771753128))
Uniform_1000	a	5	ConfidenceInterval(low=np.float64(5.00906670215289), high=np.float64(5.03045982464733))
Uniform_1000	b	15	ConfidenceInterval(low=np.float64(14.972650714059084), high=np.float64(14.993640396351307))
Bernoulli_100	p	0.7	ConfidenceInterval(low=np.float64(0.51), high=np.float64(0.71))
Bernoulli_1000	p	0.7	ConfidenceInterval(low=np.float64(0.683), high=np.float64(0.7395315556494589))
Binomial_100	p	0.4	ConfidenceInterval(low=np.float64(0.40099999999999997), high=np.float64(0.449))
Binomial_100	n	20	ConfidenceInterval(low=np.float64(nan), high=np.float64(nan))
Binomial_1000	p	0.4	ConfidenceInterval(low=np.float64(0.3952), high=np.float64(0.40859597650890084))
Binomial_1000	n	20	ConfidenceInterval(low=np.float64(nan), high=np.float64(nan))
Normal_100	u	5	ConfidenceInterval(low=np.float64(4.2866010683413664), high=np.float64(5.080219560878985))
Normal_100	sigma^2	4	ConfidenceInterval(low=np.float64(3.095542099666651), high=np.float64(5.418955514740354))
Normal_1000	u	5	ConfidenceInterval(low=np.float64(4.895684444780629), high=np.float64(5.142780147486582))
Normal_1000	sigma^2	4	ConfidenceInterval(low=np.float64(3.702797583031571), high=np.float64(4.354063990643682))